Program No: 13	Roll Number: 23071A0592	Date: 29/4/25
Program Title: Page Processing		

Problem Statement: Write a program to demonstrate page Processing Algorithm.

Program:

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
   int numberOfPages, pageSize, logicalAddress;
   vector<int> pageTable(100); // Maximum 100 pages

   cout << "Enter number of pages (max 100): ";
   cin >> numberOfPages;

   cout << "Enter page size (in bytes): ";
   cin >> pageSize;

   cout << "Enter the frame number for each page (-1 if page not in memory):\n";
   for (int page = 0; page < numberOfPages; page++) {
      cout << "Page " << page << " -> Frame: ";
      cin >> pageTable[page];
   }

   cout << "Enter a logical address (0 - " << (numberOfPages * pageSize) - 1 << "): ";
   cin >> logicalAddress;

   int pageNumber = logicalAddress / pageSize;
   int offset = logicalAddress % pageSize;

   if (pageNumber >= numberOfPages) {
      cout << "Invalid logical address! Page number " << pageNumber << " is out of bounds."
<< endl;
      return 1;
   }

   if (pageTable[pageNumber] == -1) {
      cout << "Page fault! Page " << pageNumber << " is not in memory." << endl;
      return 1;
   }
```

```
    int frameNumber = pageTable[pageNumber];
    int physicalAddress = frameNumber * pageSize + offset;

    cout << "\n--- Address Translation ---" << endl;
    cout << "Logical Address: " << logicalAddress << endl;
    cout << "Page Number: " << pageNumber << endl;
    cout << "Offset: " << offset << endl;
    cout << "Mapped to Frame: " << frameNumber << endl;
    cout << "Physical Address: " << physicalAddress << endl;

    return 0;
}
```

OUTPUT (1):

```
Enter number of pages (max 100): 6
Enter page size (in bytes): 10
Enter the frame number for each page (-1 if page not in memory):
Page 0 -> Frame: 10
Page 1 -> Frame: 25
Page 2 -> Frame: 4
Page 3 -> Frame: 2
Page 4 -> Frame: 6
Page 5 -> Frame: 4
Enter a logical address (0 - 59): 70
Invalid logical address! Page number 7 is out of bounds.


=== Code Exited With Errors ===
```

OUTPUT (2):

```
Enter number of pages (max 100): 6
Enter page size (in bytes): 6
Enter the frame number for each page (-1 if page not in memory):
Page 0 -> Frame: 10
Page 1 -> Frame: 20
Page 2 -> Frame: 55
Page 3 -> Frame: 22
Page 4 -> Frame: 5
Page 5 -> Frame: 7
Enter a logical address (0 - 35): 20

--- Address Translation ---
Logical Address: 20
Page Number: 3
Offset: 2
Mapped to Frame: 22
Physical Address: 134


=== Code Execution Successful ===
```